

Course Project
Programmable Processor
TCES330 Digital System Design
Spring 2026

Authors:

Student A: Seth Amico

Student B: John R Teal Jr.

Submission Date: June 10, 2026

Contents

Project Overview	3
1. Requirements	3
1.1 Requirements for Controller.....	4
1.2 Requirements for Datapath	4
1.3 Requirements for Processor.....	4
1.4 Requirements for Project.....	5
2. Design	5
2.1 Controller.sv.....	5
2.2 Datapath.sv.....	8
2.3 Processor.sv	9
2.4 Project.sv.....	10
3. Test Procedures and Results	11
3.1 FSM_tb	11
3.2 Controller_tb.....	13
3.3 Datapath_tb	13
3.4 testProcessor	14
3.5 Additional Module-Level Tests	15
4. DE Board Test Results.....	17
5. Conclusion	18
6. References.....	19
Appendix.....	20

Project Overview

The purpose of this course project was to design, simulate, integrate, and test a small programmable processor using SystemVerilog. The processor was built from several smaller modules, including a finite state machine, program counter, instruction register, ROM, RAM, register file, ALU, controller, datapath, and top-level board interface. By completing the project in stages, the design process reinforced how individual digital system components work together to form a functioning processor. The final processor was verified through ModelSim simulations and then tested on the DE10 board using manual clock stepping, synchronous reset, switch-controlled debug output, and seven-segment displays.

Our team members for this project were Seth Amico and John Teal. We managed the workload by dividing the processor into smaller design and verification tasks, then combining those pieces through several stages of integration. Seth contributed to the early project structure and module development, including the initial datapath work, RAM and ROM setup, control unit development, program counter work, and early integration between the controller-related modules. John contributed to module refinement, interface corrections, FSM updates, datapath debugging, testbench creation, simulation setup, processor integration, top-level Project.sv board integration, Decoder and KeyFilter support, Quartus cleanup, timing constraints, and DE10 board testing. Both team members contributed to debugging and integration as the design moved from individual modules to the final processor and board-level demonstration.

1. Requirements

The project required the design, simulation, integration, and board testing of a small programmable processor in SystemVerilog. The required instruction set included ADD, SUB, LOAD, STORE, HALT, and NOOP. The processor was divided into two primary modules: the

Controller and the Datapath. These were then connected in Processor.sv and mapped to the DE10 board through the top-level Project.sv file. The ROM and RAM were preloaded with test instructions and data, and the completed processor was verified using ModelSim simulations and DE10 board testing. Optional extra credit instructions included JNZ and JLT.

1.1 Requirements for **Controller**

The Controller was required to manage instruction fetch, instruction decode, and control-signal generation. It included the finite state machine (FSM), Program Counter (PC), Instruction Register (IR), and Instruction Memory (ROM). The PC requested the instruction address from the ROM, which wrote the instruction to the IR, where the FSM could decode the instruction data into control signals for the Datapath.

1.2 Requirements for **Datapath**

The Datapath was required to perform memory access, register movement, and ALU operations. It included the Data Memory (RAM), Register File (RF), ALU, and source-select mux. The Controller provided the Datapath control signals, including RAM address, RAM write enable, Register File addresses, Register File write enable, mux select, ALU function select.

1.3 Requirements for **Processor**

The Processor Module was required to connect the Controller and Datapath into one complete system. It exposed key internal signals for simulation and testing, including the PC, IR, ALU inputs, ALU outputs, current FSM state, and next FSM state. It also included clock and ResetN inputs to control processor operation.

1.4 Requirements for **Project**

The Project Module was required to connect the processor to the DE10 Board. The design used KEY [1] and KEY [2] for manual processor control, while KEY[0] was not used in the final DE10 board implementation. One key was used as the manual clock input, and the other was used as ResetN. A KeyFilter module was used on the manual clock input to reduce button debounce. Decoder modules converted binary values into seven-segment HEX display outputs. Switches SW [9:7] were used to select which internal processor value appeared on the HEX displays, including the IR, PC, NextState, State, ALU inputs, ALU output, and two blank reserved slots.

2. Design

The processor design was divided into four major SystemVerilog sections: The Controller, the Datapath, the Processor integration module, and the top-level Project module for the DE10 Board. The Controller is responsible for fetching and decoding instructions, while the Datapath performs data movement, memory access, and ALU operations. The Processor module connects the Controller and Datapath into one complete processor, and Project.sv connects the processor to the physical switches, keys, and HEX displays on the DE10 board.

2.1 Controller.sv

The Controller contains the FSM, Program Counter (PC), Instruction Register (IR), and the Instruction Memory (ROM). The PC supplies the ROM address, the ROM outputs the current 16-bit instruction, and the IR stores that instruction for the FSM to decode. After decoding the FSM generates the control signals needed by the Datapath.

The FSM controls the Data Memory address, Data Memory write enable, Register File writeback source, Register File read/write addresses, Register File write enable, and ALU function select. It also receives the ALU flags Z, N, and V from the Datapath. These flags are used for conditional jump instructions such as JNZ and JLT.

The FSM begins in INIT after reset, then moves through FETCH and DEC before branching to the correct instruction state. ADD and SUB perform ALU operations, LOAD moves RAM data into the Register File, STORE writes Register File data into RAM, NOOP returns to FETCH without changing data, and HALT holds the processor in the halt state. Figure 1 shows the Controller layout, and Figure 2 shows the simplified FSM flowchart.

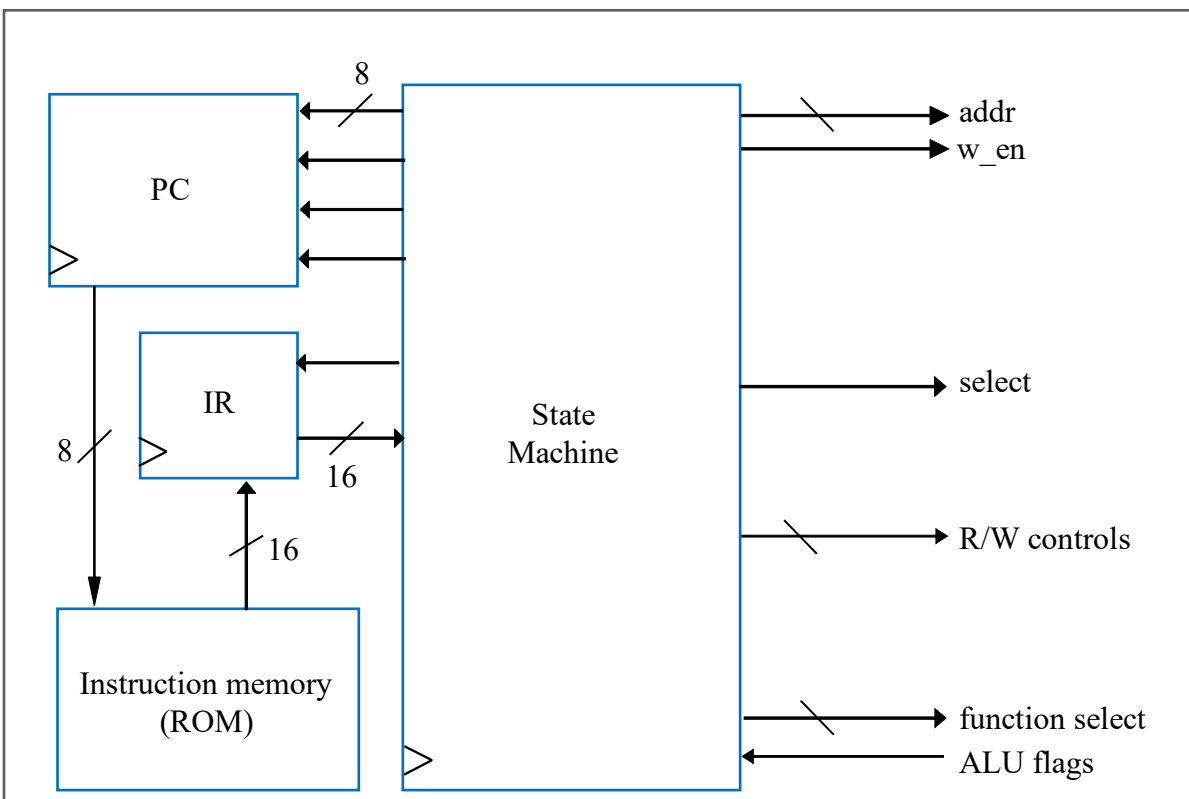


Figure 1 Controller module diagram showing the State Machine, Program Counter, Instruction Register, and Instruction Memory. Created by Seth Amico, June 4, 2026, University of Washington.

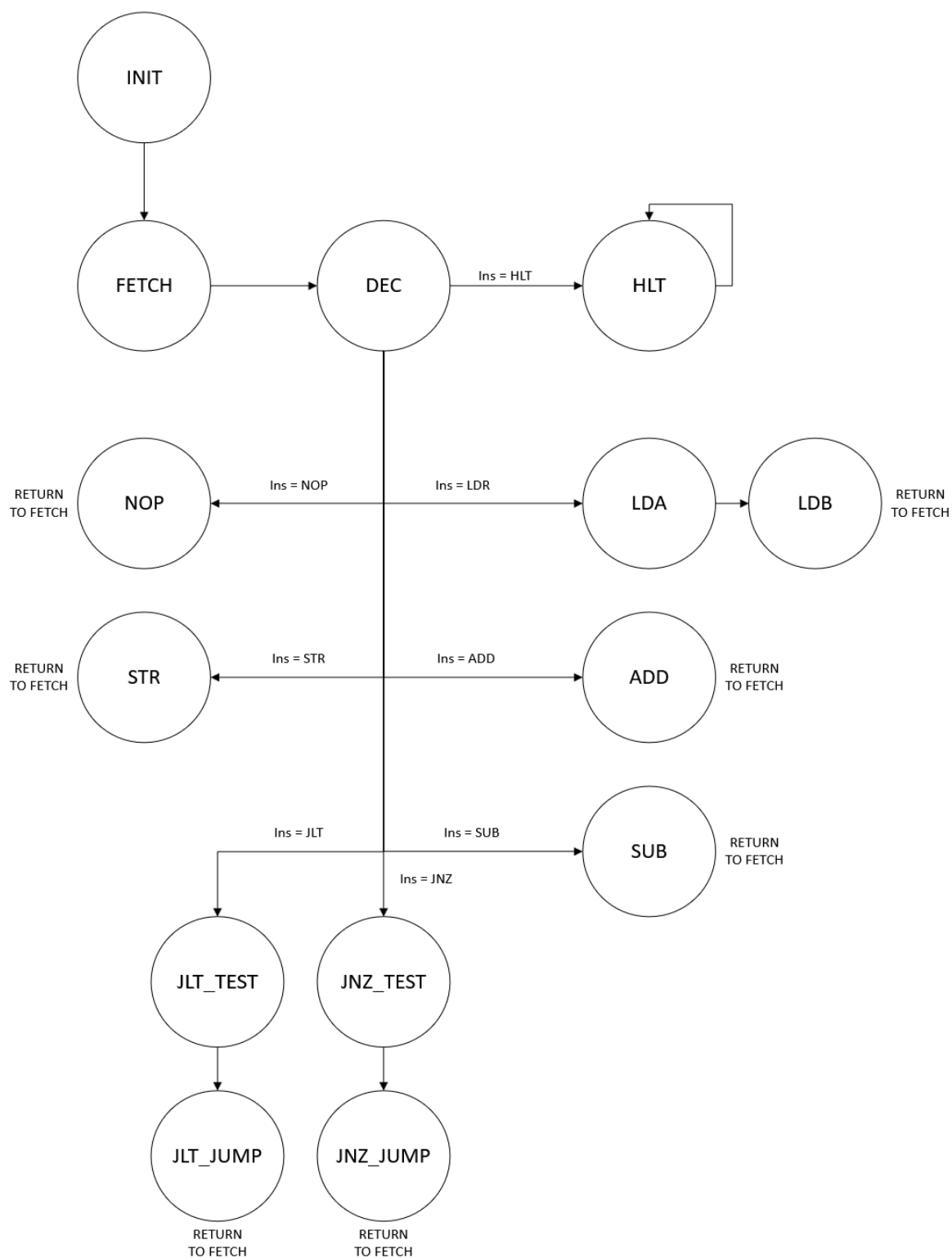


Figure 2 Simplified FSM flowchart showing state transitions. Created by Seth Amico and John R Teal Jr., June 4, 2026, University of Washington.

2.2 Datapath.sv

The Datapath module is one of the two critical sections of the processor. It houses the Arithmetic Logic Unit (ALU), the Register File (RF) and the Data Memory (RAM). The Data Memory acts as the Random Access Memory (RAM) of the processor, holding 256 16-bit values. It provides data to the Register File during LOAD operations and receives data from the Register File during STORE operations. The Register File contains sixteen 16-bit registers, allowing two simultaneous combinational read operations and one synchronous write operation. Both RF output ports are wired to the ALU inputs, and the A-side register output is also used as the data input to RAM during STORE operations. The RF receives write data from either the ALU output or the RAM output. The ALU submodule reads data from two 16-bit input register buses and combinationally performs one of eight operations based on the 3-bit function select bus. Additionally, it sets three output flags based on the result of the operation. These three modules are augmented by a 16-bit two-to-one mux, which selects the input of the RF from the ALU output or the RAM output.

The Datapath module interfaces with the Controller through several control and data signals. The RAM submodule exposes an 8-bit data address line, a write enable line, and a global clock input. The control mux exposes its select line, with a logic high selecting the RAM as a source. The RF submodule exposes three 4-bit address select lines: two read addresses and one write address. These address lines are joined by a write enable line and clock input. The ALU submodule exposes a 3-bit function select line, as well as three individual flags: Z, N, and V.

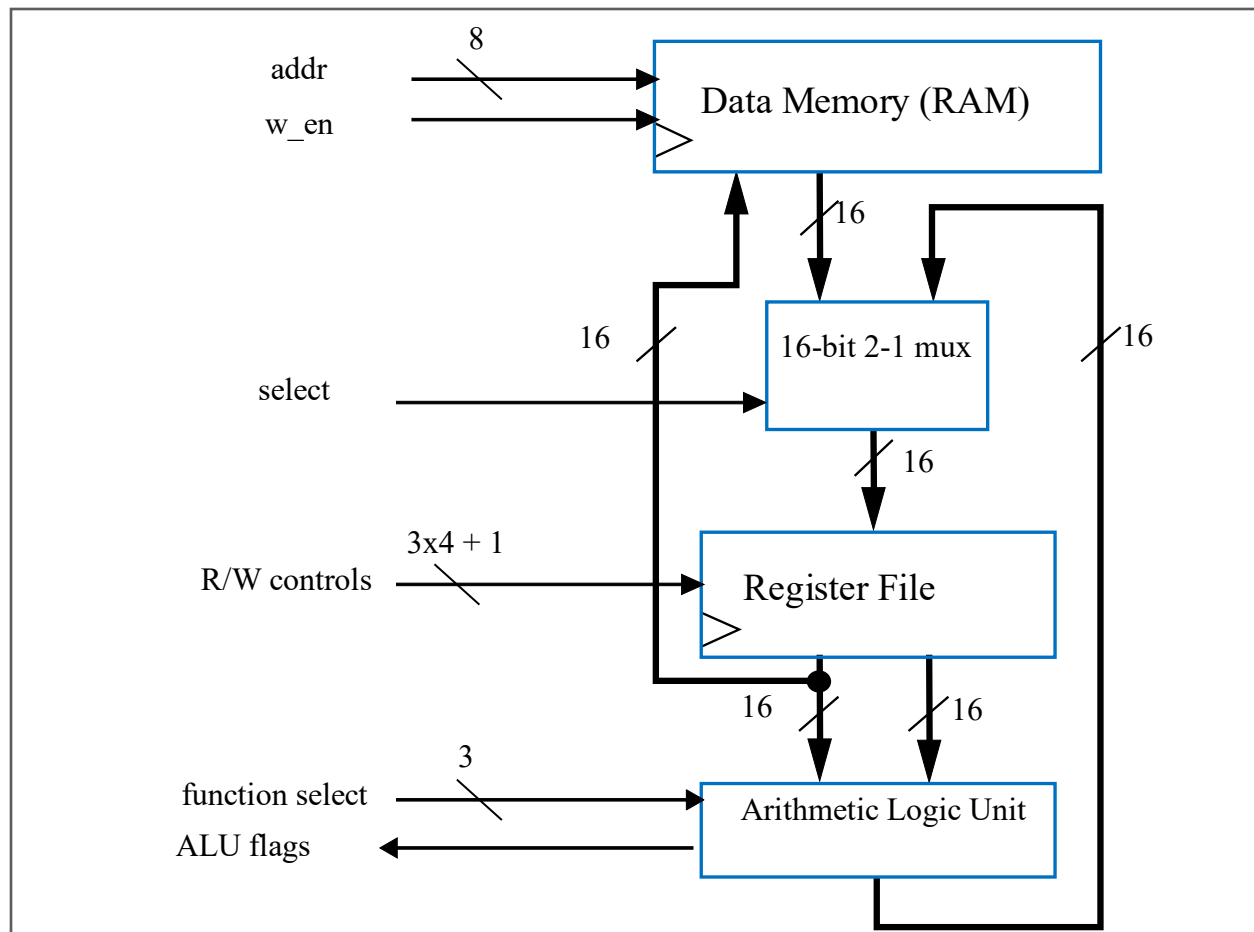


Figure 3: Datapath module flowchart depicting Data Memory, source mux, Register file, and ALU. Uses materials from TCES 330 slide deck. Seth Amico, 6/4/2026. University of Washington

2.3 Processor.sv

The Processor module connects the Controller and Datapath into one complete processor. Internal wires carry the Controller outputs into the Datapath, including the RAM address, RAM write enable, Register File controls, mux select, and ALU function select. The design was extended with ALU flag signals for optional conditional jump support, allowing the FSM to make conditional decisions when those instructions are tested.

The Processor module also exposes key internal signals for simulation and board testing. These include the PC, IR, ALU inputs, ALU outputs, current FSM state, and next FSM state.

These outputs made it easier to verify the processor in ModelSim and display selected values on the DE10 board. Figure 4 shows the combined Controller and Datapath layout.

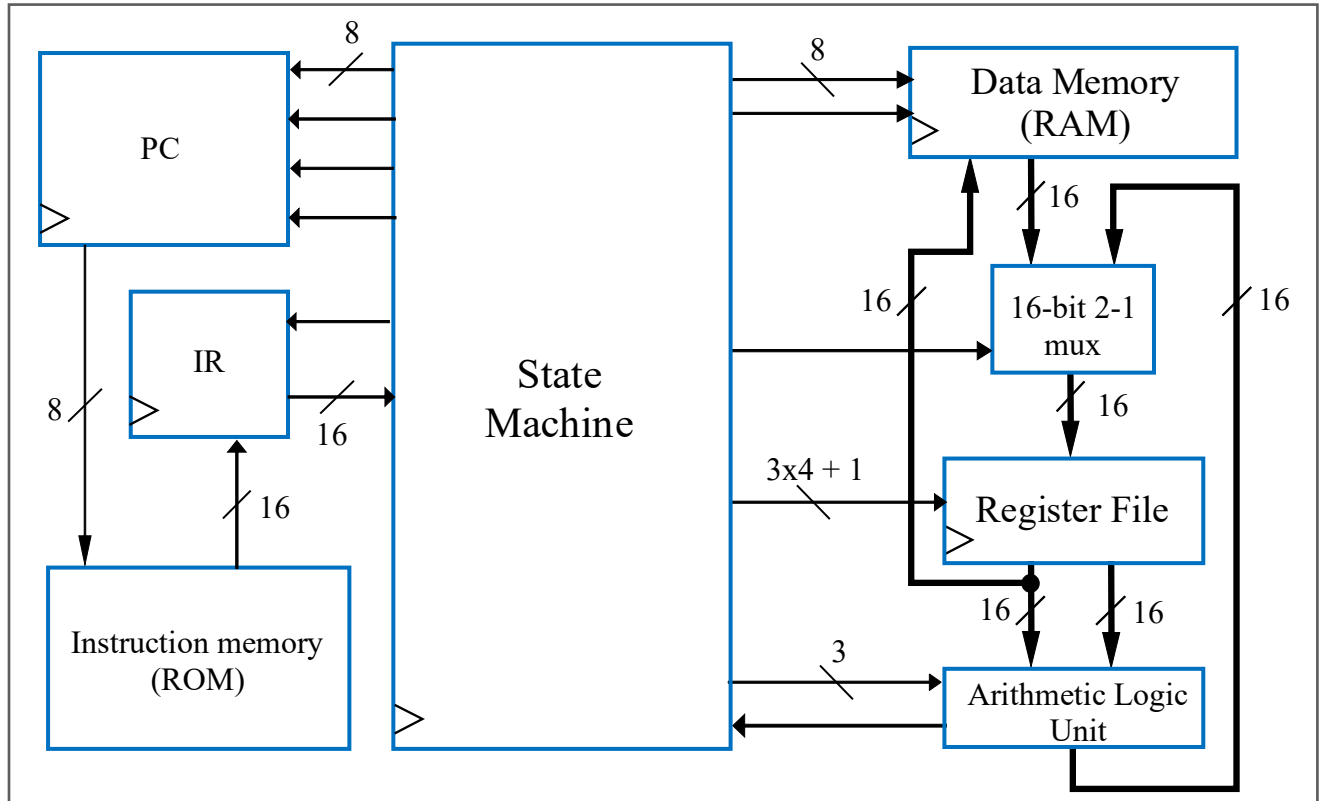


Figure 4: Combined processor layout showing the connection between the Controller and Datapath. Created by Seth Amico, June 4, 2026, University of Washington.

2.4 Project.sv

Project.sv is the top-level module used for the DE10 board testing. It instantiates the Processor, KeyFilter, and Decoder modules. KEY [1] is used as ResetN, while KEY [2] is passed through the KeyFilter and used as a manual clock. The KeyFilter reduces button bounce so that one button press is less likely to create multiple clock pulses.

The Decoder modules convert 4-bit binary values into 7-segment HEX to display outputs. SW [9:7] selects which internal processor value is shown on the HEX displays, as shown in Table 1: mapping of SW values to HEX display data.

Table 1: mapping of SW values to HEX display data.

Created by Seth Amico, June 5, 2026, University of Washington

SW[9:7]	HEX displayed value
000	Instruction Register
001	Program Counter
010	NextState, State
011	ALU input A
100	ALU input B
101	ALU output

The display options include IR_Out, PC_Out, combined State and NextState information, held ALU input A, held ALU input B, and held ALU output. The held ALU values were used so the final meaningful arithmetic values could still be displayed after the processor reached HALT. This allowed the processor to be stepped manually and observed on the board during testing.

3. Test Procedures and Results

The processor modules were verified using SystemVerilog testbenches in ModelSim. Each testbench was used to confirm that the module met its design requirements before the full processor was tested as an integrated system. The simulations checked state transitions, control signals, memory/register operations, ALU outputs, and processor-level execution.

Simulation results are included for each testbench. These results are used to show whether the tested modules satisfied the requirements described in Section 1.

3.1 FSM_tb

The FSM testbench verified that the FSM moved through the correct states and produced the expected control outputs. The testbench used three SystemVerilog tasks to organize the simulation. The first task, `check_value()`, compared an output signal against its expected value and recorded whether the test passed or failed. The second task, `reset_fsm_with_instruction()`, reset the FSM, loaded a selected instruction, and verified that the FSM restarted correctly by

moving through the INIT, FETCH, and DEC states. This placed the FSM one clock cycle before the execution state being tested. The third task, tick(), advanced the simulation to the next positive clock edge.

Each FSM state was tested by first resetting the FSM with a relevant instruction. The simulation was then advanced one clock cycle so the FSM moved from DEC into the target execution state. The control outputs were compared against the expected values using check_value(). Passing checks were added to the total pass count, while failed checks displayed a debug message showing the expected and actual values. After each state was checked, the simulation advanced one additional clock cycle to confirm that the FSM returned to FETCH when expected.

```
VSIM 7> run -all
# Starting FSM testbench.
# Testing NOOP.
# Testing STORE.
# Testing LOAD.
# Testing ADD.
# Testing SUB.
# Testing HALT.
# Testing JMP.
# Testing JNZ taken.
# Testing JNZ not taken.
# Testing JLT taken.
# Testing JLT not taken.
# FSM testbench complete.
# Passes: 118
# Failures: 0
# RESULT: PASS
# ** Note: $stop      : FSM_tb.sv(363)
#    Time: 506 ps  Iteration: 0  Instance: /FSM_tb
# Break in Module FSM_tb at FSM_tb.sv line 363
```

Figure 5 ModelSim simulation output for the FSM testbench. Created by John R. Teal Jr., June 5, 2026, University of Washington.

3.2 Controller_tb

The Controller testbench verified the basic operation of the control unit over multiple clock cycles. The simulation stepped the Controller through 34 cycles with blank input conditions and printed the output values for each cycle. These outputs were used to observe the Controller state progression, PC behavior, IR output, and generated datapath control signals.

Because the FSM testbench already verified the detailed state transitions and control outputs, this testbench was kept simple. Its purpose was to confirm that the full Controller module could run continuously and produce stable output behavior when its submodules were connected together.

```
VSIM 17> run -all
# Starting Control Unit testbench.
# cycle=0 PC=00 IR_ld=1 IR_in=21ba IR=xxxx State=1 Next=2 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=1 PC=01 IR_ld=0 IR_in=21ba IR=21ba State=2 Next=5 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=2 PC=01 IR_ld=0 IR_in=22ab IR=21ba State=5 Next=6 D_Addr=1b D_wr=0 RF_s=1 RF_W_addr=a RF_W_en=0 Alu_s0=000
# cycle=3 PC=01 IR_ld=0 IR_in=22ab IR=21ba State=6 Next=1 D_Addr=1b D_wr=0 RF_s=1 RF_W_addr=a RF_W_en=1 Alu_s0=000
# cycle=4 PC=01 IR_ld=1 IR_in=22ab IR=21ba State=1 Next=2 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=5 PC=02 IR_ld=0 IR_in=22ab IR=22ab State=2 Next=5 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=6 PC=02 IR_ld=0 IR_in=4aba IR=22ab State=5 Next=6 D_Addr=2a D_wr=0 RF_s=1 RF_W_addr=b RF_W_en=0 Alu_s0=000
# cycle=7 PC=02 IR_ld=0 IR_in=4aba IR=22ab State=6 Next=1 D_Addr=2a D_wr=0 RF_s=1 RF_W_addr=b RF_W_en=1 Alu_s0=000
# cycle=8 PC=02 IR_ld=1 IR_in=4aba IR=22ab State=1 Next=2 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=9 PC=03 IR_ld=0 IR_in=4aba IR=4aba State=2 Next=8 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=10 PC=03 IR_ld=0 IR_in=23cb IR=4aba State=8 Next=1 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=a RF_W_en=1 Alu_s0=010
# cycle=11 PC=03 IR_ld=1 IR_in=23cb IR=4aba State=1 Next=2 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=12 PC=04 IR_ld=0 IR_in=23cb IR=23cb State=2 Next=5 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=13 PC=04 IR_ld=0 IR_in=3aba IR=23cb State=5 Next=6 D_Addr=3c D_wr=0 RF_s=1 RF_W_addr=b RF_W_en=0 Alu_s0=000
# cycle=14 PC=04 IR_ld=0 IR_in=3aba IR=23cb State=6 Next=1 D_Addr=3c D_wr=0 RF_s=1 RF_W_addr=b RF_W_en=1 Alu_s0=000
# cycle=15 PC=04 IR_ld=1 IR_in=3aba IR=23cb State=1 Next=2 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=16 PC=05 IR_ld=0 IR_in=3aba IR=3aba State=2 Next=7 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=17 PC=05 IR_ld=0 IR_in=27eb IR=3aba State=7 Next=1 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=a RF_W_en=1 Alu_s0=001
# cycle=18 PC=05 IR_ld=1 IR_in=27eb IR=3aba State=1 Next=2 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=19 PC=06 IR_ld=0 IR_in=27eb IR=27eb State=2 Next=5 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=20 PC=06 IR_ld=0 IR_in=4aba IR=27eb State=5 Next=6 D_Addr=7e D_wr=0 RF_s=1 RF_W_addr=b RF_W_en=0 Alu_s0=000
# cycle=21 PC=06 IR_ld=0 IR_in=4aba IR=27eb State=6 Next=1 D_Addr=7e D_wr=0 RF_s=1 RF_W_addr=b RF_W_en=1 Alu_s0=000
# cycle=22 PC=06 IR_ld=1 IR_in=4aba IR=27eb State=1 Next=2 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=23 PC=07 IR_ld=0 IR_in=4aba IR=4aba State=2 Next=8 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=24 PC=07 IR_ld=0 IR_in=1a6a IR=4aba State=8 Next=1 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=a RF_W_en=1 Alu_s0=010
# cycle=25 PC=07 IR_ld=1 IR_in=1a6a IR=4aba State=1 Next=2 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=26 PC=08 IR_ld=0 IR_in=1a6a IR=1a6a State=2 Next=4 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=27 PC=08 IR_ld=0 IR_in=5000 IR=1a6a State=4 Next=1 D_Addr=6a D_wr=1 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=28 PC=08 IR_ld=1 IR_in=5000 IR=1a6a State=1 Next=2 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=29 PC=09 IR_ld=0 IR_in=5000 IR=5000 State=2 Next=9 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=30 PC=09 IR_ld=0 IR_in=0000 IR=5000 State=9 Next=9 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=31 PC=09 IR_ld=0 IR_in=0000 IR=5000 State=9 Next=9 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=32 PC=09 IR_ld=0 IR_in=0000 IR=5000 State=9 Next=9 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# cycle=33 PC=09 IR_ld=0 IR_in=0000 IR=5000 State=9 Next=9 D_Addr=00 D_wr=0 RF_s=0 RF_W_addr=0 RF_W_en=0 Alu_s0=000
# Control Unit testbench complete.
# ** Note: $stop : Control_Unit_tb.sv(113)
# Time: 356 ps Iteration: 0 Instance: /Control_Unit_tb
# Break in Module Control_Unit_tb at Control_Unit_tb.sv line 113
```

Figure 6 ModelSim simulation output for the Controller testbench. Created by John R. Teal Jr., June 5, 2026, University of Washington.

3.3 Datapath_tb

The Datapath testbench used four tasks to reduce repeated code and make the simulation easier to follow. Check16() task compared a 16-bit output value against an expected value,

logged the result, and reported any errors to the console. The `load_ram_to_reg()` task loaded data from RAM into a selected register and used `check16()` to verify that the loaded value matched the expected value.

The `alu_write_reg()` task selected two Register File read addresses, sent those values to the ALU, selected the ALU as the Register File write source, and wrote the ALU result into a destination register. The ALU output and written register value were then compared against the expected result. The `store_reg_to_ram()` task selected a Register File output, passed it through the Datapath, and wrote the value into a selected RAM address. The RAM output was then checked with `check16()`.

Together these tasks simulated the Datapath control signals used during LOAD, STORE, and ALU-based instructions. The result confirmed that the Datapath could move data between RAM and the Register File, perform ALU operations, and store results correctly.

```
VSIM 31> run -all
# Starting Datapath testbench.
# Datapath testbench complete.
# Passes: 11
# Failures: 0
# RESULT: PASS
# ** Note: $stop      : Datapath_tb.sv(249)
#    Time: 126 ps   Iteration: 0   Instance: /Datapath_tb
# Break in Module Datapath_tb at Datapath_tb.sv line 249
```

Figure 7 ModelSim simulation output for the Datapath testbench. Created by John R. Teal Jr., June 5, 2026, University of Washington.

3.4 testProcessor

The provided processor testbench, `testProcessor.sv`, instantiates the completed Processor module and connects its clock, reset, IR, PC, state, and ALU output signals. This testbench was important because it verified the full processor after the Controller and Datapath were connected together. During simulation, the processor stepped through the instruction sequence stored in

ROM while printing the PC, IR, FSM state, ALU inputs, ALU output, and register address information to the ModelSim console. The simulation showed the processor loading data from RAM, performing the required SUB, ADD, and SUB operations, storing the final result, and reaching the HALT instruction 5000. Reaching the HALT instruction caused the testbench to end normally, confirming that the complete processor executed the required program sequence correctly.

```

VSIM 44> run -all
#
# Begin Simulation.
# Time is 0 : ResetN = 0 PC_Out = xx IR_Out = xxxx State = x ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 10000 : ResetN = 0 PC_Out = xx IR_Out = xxxx State = 0 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 30000 : ResetN = 0 PC_Out = 00 IR_Out = xxxx State = 0 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 40000 : ResetN = 1 PC_Out = 00 IR_Out = xxxx State = 0 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 50000 : ResetN = 1 PC_Out = 00 IR_Out = xxxx State = 1 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 70000 : ResetN = 1 PC_Out = 01 IR_Out = 21ba State = 2 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 90000 : ResetN = 1 PC_Out = 01 IR_Out = 21ba State = 5 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 110000 : ResetN = 1 PC_Out = 01 IR_Out = 21ba State = 6 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 130000 : ResetN = 1 PC_Out = 01 IR_Out = 21ba State = 1 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 150000 : ResetN = 1 PC_Out = 02 IR_Out = 22ab State = 2 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 170000 : ResetN = 1 PC_Out = 02 IR_Out = 22ab State = 5 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 190000 : ResetN = 1 PC_Out = 02 IR_Out = 22ab State = 6 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 210000 : ResetN = 1 PC_Out = 02 IR_Out = 22ab State = 1 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 230000 : ResetN = 1 PC_Out = 03 IR_Out = 4aba State = 2 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 250000 : ResetN = 1 PC_Out = 03 IR_Out = 4aba State = 8 ALU A = 21ba ALU B = a04e ALU Out = 816c RA Address = 1010
# Time is 270000 : ResetN = 1 PC_Out = 03 IR_Out = 4aba State = 1 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 290000 : ResetN = 1 PC_Out = 04 IR_Out = 23cb State = 2 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 310000 : ResetN = 1 PC_Out = 04 IR_Out = 23cb State = 5 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 330000 : ResetN = 1 PC_Out = 04 IR_Out = 23cb State = 6 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 350000 : ResetN = 1 PC_Out = 04 IR_Out = 23cb State = 1 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 370000 : ResetN = 1 PC_Out = 05 IR_Out = 3aba State = 2 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 390000 : ResetN = 1 PC_Out = 05 IR_Out = 3aba State = 7 ALU A = 816c ALU B = 71ac ALU Out = f318 RA Address = 1010
# Time is 410000 : ResetN = 1 PC_Out = 05 IR_Out = 3aba State = 1 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 430000 : ResetN = 1 PC_Out = 06 IR_Out = 27eb State = 2 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 450000 : ResetN = 1 PC_Out = 06 IR_Out = 27eb State = 5 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 470000 : ResetN = 1 PC_Out = 06 IR_Out = 27eb State = 6 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 490000 : ResetN = 1 PC_Out = 06 IR_Out = 27eb State = 1 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 510000 : ResetN = 1 PC_Out = 07 IR_Out = 4aba State = 2 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 530000 : ResetN = 1 PC_Out = 07 IR_Out = 4aba State = 8 ALU A = f318 ALU B = b17f ALU Out = 4199 RA Address = 1010
# Time is 550000 : ResetN = 1 PC_Out = 07 IR_Out = 4aba State = 1 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 570000 : ResetN = 1 PC_Out = 08 IR_Out = 1a6a State = 2 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
# Time is 590000 : ResetN = 1 PC_Out = 08 IR_Out = 1a6a State = 4 ALU A = 4199 ALU B = xxxx ALU Out = 4199 RA Address = 1010
# Time is 610000 : ResetN = 1 PC_Out = 08 IR_Out = 1a6a State = 1 ALU A = xxxx ALU B = xxxx ALU Out = xxxx RA Address = 0000
#
# End of Simulation.
#
# ** Note: $stop : testProcessor.sv(32)
# Time: 630 ns Iteration: 2 Instance: /testProcessor
# Break in Module testProcessor at testProcessor.sv line 32

```

Figure 8 ModelSim simulation output from the professor-provided testProcessor testbench. Testbench designed by Professor Sheng; simulation output generated by John R. Teal Jr., June 5, 2026, University of Washington.

3.5 Additional Module-Level Tests

The PC testbench was included as an additional module-level test to verify the program counter before it was integrated into the Control Unit. This test confirmed that the PC cleared correctly during reset and advanced correctly when the increment control signal was active.

Verifying the PC separately helped ensure that instruction sequencing would work properly once the PC was connected to the ROM and FSM.

The IR testbench was used to verify the behavior of the instruction register. This test confirmed that the IR loaded a new instruction only when the load signal was asserted and held its previous instruction value when the load signal was inactive. This was important because the FSM depends on a stable instruction value during the decode and execution states.

The ROM testbench was used to confirm that the instruction memory contained the correct program instructions at the expected addresses. The test verified that the ROM output matched the instruction sequence used by the processor, including the LOAD, ADD, SUB, STORE, and HALT instructions. This helped confirm that any later processor behavior came from the intended program and not from incorrect instruction memory contents.

The RAM testbench was used to verify that the data memory contained the correct initial project values. The test checked that address 1B contained 21BA, address 2A contained A04E, 3C contained 71AC, and 7E contained B17F. These memory values were required for the processor to compute the final expected result of 4199.

```

# cycle=15 PC=04 IR=23cb State=1 Next=2 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=16 PC=05 IR=3aba State=2 Next=7 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=17 PC=05 IR=3aba State=7 Next=1 ALU_A=816c ALU_B=71ac ALU_Out=f318 D_Addr=00 D_wr=0
# cycle=18 PC=05 IR=3aba State=1 Next=2 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=19 PC=06 IR=27eb State=2 Next=5 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=20 PC=06 IR=27eb State=5 Next=6 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=7e D_wr=0
# cycle=21 PC=06 IR=27eb State=6 Next=1 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=7e D_wr=0
# cycle=22 PC=06 IR=27eb State=1 Next=2 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=23 PC=07 IR=4aba State=2 Next=8 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=24 PC=07 IR=4aba State=8 Next=1 ALU_A=f318 ALU_B=b17f ALU_Out=4199 D_Addr=00 D_wr=0
# cycle=25 PC=07 IR=4aba State=1 Next=2 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=26 PC=08 IR=1a6a State=2 Next=4 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=27 PC=08 IR=1a6a State=4 Next=1 ALU_A=4199 ALU_B=xxxx ALU_Out=4199 D_Addr=6a D_wr=1
# cycle=28 PC=08 IR=1a6a State=1 Next=2 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=29 PC=09 IR=5000 State=2 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=30 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=31 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=32 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=33 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=34 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=35 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=36 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=37 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=38 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=39 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=40 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=41 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=42 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=43 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=44 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=45 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=46 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=47 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=48 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# cycle=49 PC=09 IR=5000 State=9 Next=9 ALU_A=xxxx ALU_B=xxxx ALU_Out=xxxx D_Addr=00 D_wr=0
# Processor testbench complete.
# Passes: 4
# Failures: 0
# RESULT: PASS
# ** Note: $stop : Processor_tb.sv(150)
# Time: 516 ps Iteration: 0 Instance: /Processor_tb
# Break in Module Processor_tb at Processor_tb.sv line 150

```

Figure 9 Console output from the custom ModelSim Processor_tb testbench. Created by John R. Teal Jr., June 5, 2026, University of Washington.

The custom Processor testbench was used as an additional full-system check beyond the professor-provided testProcessor testbench. This testbench monitored the processor as it executed the full instruction sequence and checked that the STORE instruction used the correct final value from register A. After updating the expected result to match the assigned project data, the custom processor testbench confirmed that the final stored value was 4199.

4. DE Board Test Results

The completed processor was tested on the DE10 board after successful Quartus compilation and programming. The board test used manual clock stepping so each processor cycle could be observed one step at a time. KEY [1] returned the processor to its starting state,

and KEY [2] advanced the processor through the loaded ROM instructions using the KeyFilter output as the manual processor clock. Switches were used to select which internal processor value appeared on the HEX displays.

During testing, the HEX displays were used to observe IR_Out, PC_Out, current FSM state, next FSM state information, held ALU input values, and held ALU output values. This allowed the processor's behavior to be checked against the expected instruction sequence. The board output matched the expected behavior from the ModelSim simulations, showing that the processor could fetch instructions, move through the FSM states, perform ALU operations, store the final result, and display internal debug values correctly.

Several Quartus warnings appeared during the compilation. Some warnings were caused by unused board inputs or outputs, since not every DE10 switch, key, LED, or display was needed for the project. These warnings were solved by trimming the imported pin assignments to strictly the used connections. The project also required checking the pin assignments and voltage settings to make sure the board I/O used the correct 3.3 V standard. After the pin assignments and top-level module connections were corrected, the design was compiled and programmed successfully. During the final board test, the processor produced the expected final result of 4199. This value was visible through the held ALU output display mode after the final SUB operation and through the held ALU_A display mode during the STORE operation.

5. Conclusion

Overall, this project gave us a stronger understanding of hierarchical module-based SystemVerilog design through the creation of a programmable processor. When designing the Controller, we gained a better understanding of FSM design through the state diagram creation

and timing analysis through our PC/IR implementation. Our design of the RAM and ROM used a predesigned, parameterized module from Quartus' IP Catalog which we then used a Memory Initialization File (.mif) to initialize values into. Our design of the datapath required many iterations and created an opportunity to rapidly move through multiple improved designs. As we developed the project, we learned numerous intersecting facets of the team-centered design method and the iterative process. Through the early phases of processor design, we created multiple modules—one for each of the subsections. This method proved invaluable for testing and error correction. If a bug was found within one module, it could easily be fixed without needing to update the entire project. Seth created a [GitHub repository](#) for this project, which we found effective for maintaining a unified project, keeping individual members up to speed, and letting us work on multiple modules asynchronously. John's work on testbenches and documentation made it easier to clean up older code, expand existing modules, and diagnose issues during integration. We found that while both of us were capable of adding to the project on our own, working in tandem in person or over voice call let us catch problems and understand the project faster and to a more complete degree.

6. References

- ModelSim Download Page*. 4 June 2026. <<https://www.altera.com/downloads/simulation-tools/modelsim-fpgas-standard-edition-software-version-20-1-1>>.
- Quartus Prime Download Page*. 4 June 2026. <<https://www.altera.com/products/development-tools/quartus>>.
- Sheng, Jenny. "TCES330 Course Materials." 04 June 2026. *University of Washington Canvas*. <<https://canvas.uw.edu/courses/1894518/files>>.
- Stephen Brown, Zvonko Vranesic. *Fundamentals of Digital Logic with Verilog Design, 3rd Ed.* New York, NY: McGraw-Hill, 2014.
- Sutherland, Stuart. *Verilog HDL Quick Reference Guide*. Tualatin, OR: Sutherland HDL, Inc, 2007.
- Terasic. *DE10 Standard User Manual*. Terasic Inc., 2017.

Appendix

The project offered extra credit opportunities by allowing us to implement two new instructions: Jump if less than (JLT) and Jump if not zero (JNZ). These required some modifications to the processor design. First, the ALU submodule had to be modified to output state flags V (Overflow bit, for Addition operations), N (Negative, if signed), and Z (Zero). These flags are evaluated through additional FSM states, so the conditional jump instructions require separate test and jump states. One state would test the value of two registers, and the second executes the jump command if the conditions are met. Jump commands directly write to the PC to change the instruction address, so the Controller had to be changed to include a PC write enable line and a PC write data bus. The design was extended with preliminary support for the extra credit instructions, but these instructions were not fully verified in the provided processor testbench. Therefore, the submitted demonstration focused on the required base instruction sequence.